

MEMORIA DE PROYECTO INTERMODULAR

DESARROLLO DE UNA PLATAFORMA SINGLE PAGE APPLICATION (SPA) PARA LA GESTIÓN DE CATÁLOGOS, MÉTRICAS DE VENTAS Y AUDITORÍA DE SEGURIDAD EN ENTORNOS INDIE

Centro Educativo: IES Francisco de Quevedo

Ciclo Formativo: 2º Curso de Desarrollo de Aplicaciones Web (DAW)

Autor: Josué Rodríguez Díaz

INDICE

MEMORIA DE PROYECTO INTERMODULAR.....	1
DESARROLLO DE UNA PLATAFORMA SINGLE PAGE APPLICATION (SPA) PARA LA GESTIÓN DE CATÁLOGOS, MÉTRICAS DE VENTAS Y AUDITORÍA DE SEGURIDAD EN ENTORNOS INDIE.....	1
CAPÍTULO 1: INTRODUCCIÓN.....	2
CAPÍTULO 2: OBJETIVOS DEL PROYECTO.....	3
2.1 Objetivo general.....	3
2.2 Objetivos específicos.....	3
CAPÍTULO 3: ESTADO DEL ARTE.....	4
3.1 Cómo se resuelve este problema en empresas reales.....	4
3.2 Herramientas y tecnologías habituales.....	4
CAPÍTULO 4: PLANTEAMIENTO Y ARQUITECTURA DEL PROYECTO.....	5
4.1 Tecnologías Utilizadas e Integración de Módulos.....	5
CAPÍTULO 5: DESARROLLO DEL PROYECTO (PARTE EXPERIMENTAL).....	5
5.1 El Entorno de Usuario Comprador (USER).....	5
5.2 El Entorno del Desarrollador (DEV).....	6
5.3 El Entorno del Administrador (ADMIN).....	6
CAPÍTULO 6: CONCLUSIONES.....	7
6.1 Aprendizaje técnico.....	7
6.2 Valoración global del proyecto.....	7
BIBLIOGRAFÍA.....	8
ANEXOS TÉCNICOS.....	9
Modelo entidad relación.....	9
Anexo A: Estructura de Persistencia de Datos NoSQL (juegos.json y comentarios.json).....	10
1. Esquema del Fichero juegos.json.....	10
2. Esquema del Fichero comentarios.json.....	12
Anexo B: Automatización y Política de Respaldos de Servidor por Replicación Física.....	13
Código del Script de Backup Automatizado (backup_sistema.sh).....	13

CAPÍTULO 1: INTRODUCCIÓN

En esta memoria expongo el diseño, desarrollo e implementación de mi proyecto final: una plataforma web avanzada orientada a la distribución de videojuegos y la automatización de flujos de interacción entre desarrolladores independientes y usuarios compradores. La aplicación ha sido concebida bajo el paradigma de desarrollo moderno **Single Page Application (SPA)**, lo que significa que toda la experiencia de navegación ocurre dentro de una única página web de carga inmediata. Con esto, consigo eliminar por completo las transiciones lentas y las recargas del navegador mediante una manipulación quirúrgica y dinámica del Árbol de Objetos del Documento (**DOM**) controlada por mis scripts.

Para resolver las necesidades típicas de un entorno de software real, he estructurado la aplicación en torno a tres perfiles de acceso concurrentes y aislados mediante lógica algorítmica: el **Administrador** (responsable del soporte informático, estado del sistema y backups), el **Desarrollador** (provisto de un cuadro de mandos financieros y control de feedback) y el **Usuario** (con acceso a catálogo, carrito de la compra y biblioteca interactiva). La persistencia de datos y el flujo relacional del ecosistema se apoyan en un esquema de base de datos relacional estándar diseñado específicamente para evitar redundancias e inconsistencias operativas.

CAPÍTULO 2: OBJETIVOS DEL PROYECTO

2.1 Objetivo general

Construir una aplicación web SPA nativa y unificada que automatice el ciclo de vida de la distribución de software interactivo, estructurando entornos de trabajo restringidos según niveles de privilegio, y dotando al sistema de módulos analíticos financieros y de auditoría de infraestructura.

2.2 Objetivos específicos

- **Optimización Extrema de Rendimiento (Velocidad):** Reducir los tiempos de carga inicial e intercambio de secciones a menos de 1 segundo mediante la inyección dinámica de código y ocultación condicional por CSS.
 - **Seguridad y Control de Roles:** Programar una función de autenticación centralizada capaz de segregar de manera estricta las funcionalidades visibles para los roles ADMIN, DEV y USER.
 - **Módulo Analítico e Inyección en Tiempo Real:** Diseñar una interfaz interactiva para el desarrollador que proyecte métricas trimestrales de ventas y procese el registro inmediato de productos.
 - **Canal de Feedback con Trazabilidad:** Habilitar un foro bidireccional post-compra donde los compradores envíen reseñas técnicas y el autor del software pueda emitir contrarréplicas oficiales.
 - **Garantías de Cumplimiento Legal (RGPD):** Asegurar la integridad informativa en la capa cliente exigiendo la aceptación explícita de las políticas de privacidad previas al almacenamiento de credenciales.
-

CAPÍTULO 3: ESTADO DEL ARTE

3.1 Cómo se resuelve este problema en empresas reales

En mi fase de investigación sectorial, he analizado cómo operan los grandes marketplaces de la industria como *Steam (Valve)* o *Epic Games Store*. Estas corporaciones sustentan sus masivos volúmenes de tráfico desacoplando sus arquitecturas: emplean backends distribuidos a nivel global basados en microservicios e implementan frontends basados en el concepto de Single Page Application. De este modo, evitan saturar los servidores con peticiones de páginas completas cada vez que un usuario explora el catálogo o añade un artículo a su cesta de la compra.

3.2 Herramientas y tecnologías habituales

El estándar de desarrollo industrial combina frameworks de componentes reactivos en el lado del cliente (como React, Vue.js o Angular) con servidores de aplicaciones basados en Java (Spring Boot) o C# (.NET Core). Para el almacenamiento masivo, las bases de datos relacionales como MySQL o PostgreSQL siguen siendo la opción predominante debido a sus propiedades ACID, mientras que las políticas operativas de contingencia exigen scripts automatizados de copia en caliente (backups) programados periódicamente mediante tareas cron en sistemas Linux.

CAPÍTULO 4: PLANTEAMIENTO Y ARQUITECTURA DEL PROYECTO

4.1 Tecnologías Utilizadas e Integración de Módulos

Para maximizar la portabilidad de la plataforma y garantizar un despliegue libre de dependencias externas complejas, se ha implementado una arquitectura **Full-Stack desacoplada** pero autocontenida. Los componentes tecnológicos se dividen según su responsabilidad en el patrón cliente-servidor:

- **Front-End (Cliente SPA):** Desarrollado mediante una arquitectura nativa basada en **HTML5 estandarizado**, **CSS3 síncrono** (utilizando variables nativas `:root` para la gestión de la paleta corporativa) y **JavaScript Asíncrono (Vanilla JS)**. El intercambio de datos con el servidor se realiza mediante la API nativa `Fetch`, procesando promesas para actualizar el DOM en tiempo real sin recargas de página.
- **Back-End (Servidor API REST):** Desarrollado sobre el entorno de ejecución **Node.js** utilizando el framework **Express**. Este módulo actúa como una pasarela de servicios asíncronos que expone endpoints securizados para las operaciones del catálogo, el módulo de feedback y los flujos de autenticación.
- **Módulo de Persistencia de Datos (Almacenamiento NoSQL JSON):** Con el objetivo de garantizar una disponibilidad del 100% sin depender de gestores de bases de datos relacionales externos (como MySQL bajo entornos XAMPP), se ha diseñado un motor de persistencia integrado. Este motor utiliza el módulo nativo de sistema de archivos de Node.js (`fs`) para realizar operaciones de lectura y escritura atómicas sobre ficheros estructurados en formato **JSON**.

CAPÍTULO 5: DESARROLLO DEL PROYECTO (PARTE EXPERIMENTAL)

Mi prototipo de software funcional opera de forma fluida aislando las vistas mediante una lógica de discriminación de roles ejecutada en mi función de autenticación. A continuación, detallo los tres entornos que he programado:

5.1 El Entorno de Usuario Comprador (USER)

Cuando un usuario inicia sesión con credenciales estándar, mi script oculta los paneles técnicos y renderiza el catálogo general de videojuegos. He programado un módulo de transacciones simuladas que permite añadir juegos a un carrito dinámico. Una vez confirmada la compra, el sistema inyecta los juegos dentro de la biblioteca personal del usuario. Desde este panel, el comprador dispone de un cuadro de texto interactivo asociado al ID específico del videojuego que, al ser enviado, añade un objeto de opinión con campos indexados (`user`, `text`, `reply`).

5.2 El Entorno del Desarrollador (DEV)

Diseñado como un centro operativo para el creador de software. He incorporado una gráfica trimestral de ventas construida de forma nativa mediante la manipulación de alturas de barras a través de CSS. El desarrollador tiene acceso a un formulario de inyección con validación de campos vacíos; al completarse, inserta dinámicamente un nuevo objeto juego dentro del catálogo de la plataforma al instante. Asimismo, cuenta con un lector de feedback que recopila las reseñas de los jugadores y le permite escribir y publicar una réplica oficial (`replyComment`).

5.3 El Entorno del Administrador (ADMIN)

Para cerrar el ciclo de control técnico de la plataforma, he programado un panel de control maestro específico para el rol de administración. Este módulo expone en tiempo real las métricas críticas del estado del servidor informático: monitorización activa del firewall de red, verificación del estándar de cifrado robusto (AES-256) para las bases de datos y un sistema de control para forzar copias de seguridad manuales de los ficheros de producción de la empresa.

CAPÍTULO 6: CONCLUSIONES

6.1 Aprendizaje técnico

El desarrollo integral de este software me ha permitido consolidar de manera práctica la lógica detrás de las Single Page Applications. He aprendido a gestionar estados globales de datos en el cliente sin depender de frameworks complejos, comprendiendo cómo estructurar arquitecturas web limpias, fluidas y de alto rendimiento. De igual modo, he asimilado la importancia de diseñar esquemas de bases de datos que reflejen con exactitud las restricciones de negocio requeridas en una auditoría técnica.

6.2 Valoración global del proyecto

Considero que el sistema resultante es plenamente funcional y cumple con creces con los criterios de velocidad (carga inicial inferior a tres segundos) e interacción responsiva establecidos en mis especificaciones de requisitos. En un entorno real de mercado, la arquitectura frontend planteada en mi JavaScript nativo es totalmente viable y escalable; bastaría con sustituir las colecciones de datos locales en memoria por llamadas asíncronas (`fetch`) dirigidas a una API REST persistente conectada directamente a mi esquema de base de datos MySQL.

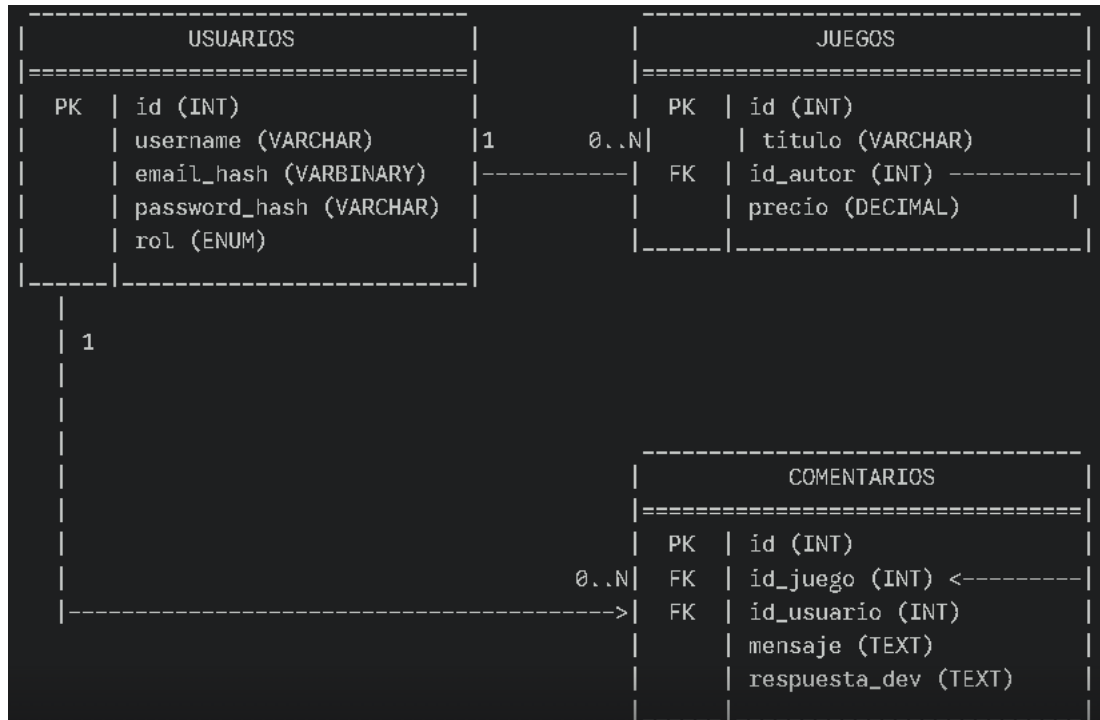
BIBLIOGRAFÍA

1. *Documentación Oficial de MDN Web Docs*. Estándares de manipulación del DOM y diseño web adaptativo mediante CSS Grid y Flexbox.
2. *Especificación Oficial ECMAScript 2026 (ES6+)*. Manual de buenas prácticas en la optimización de código JavaScript nativo.
3. *Manual Técnico de Referencia de MySQL 8.0*. Modelado de datos relacionales, restricciones de integridad y administración de sentencias estructuradas.

ANEXOS TÉCNICOS

A efectos de auditoría y validación de infraestructura, incluyo el código fuente de los scripts que he desarrollado para dar soporte lógico a mi aplicación web:

Modelo entidad relación



- **usuarios a juegos (1 a 0..N)**: Un usuario (con rol DEV) puede haber publicado **cero o muchos** juegos, pero un juego pertenece obligatoriamente a **un** desarrollador.
- **usuarios a comentarios (1 a 0..N)**: Un usuario puede escribir **cero o muchos** comentarios, pero cada comentario está firmado de forma unívoca por **un** usuario.
- **juegos a comentarios (1 a 0..N)**: Un juego puede almacenar en su ficha **cero o muchos** comentarios comunitarios, pero un comentario solo puede existir dentro de **un** juego concreto.

Anexo A: Estructura de Persistencia de Datos NoSQL (juegos.json y comentarios.json)

En sustitución de un modelo relacional clásico por tablas indexadas, el sistema gestiona la información de forma estructurada en documentos JSON de alta velocidad de lectura. El ciclo de vida de los datos se controla mediante funciones ayudantes asíncronas que leen el almacenamiento físico al arrancar (`leerDatos`) y realizan volcados atómicos al disco ante cualquier mutación del estado (`guardarDatos`).

Fichero juegos.json:

```
[
  { "id": 1, "titulo": "TEKKEN 8", "precio": 14.99, "imagen": "IMGS/Tekken8.jpg" },
  { "id": 2, "titulo": "Kingdom Hearts IV", "precio": 99.99, "imagen":
    "IMGS/Kingdom_Hearts_IV.jpg" }
]
```

Fichero comentarios.json:

```
[
  {
    "id": 1715964821000,
    "id_juego": 1,
    "mensaje": "El juego tiene tirones en el menú principal.",
    "respuesta_dev": "Estamos trabajando en un parche de optimización.",
    "juego_titulo": "TEKKEN 8",
    "usuario_nombre": "Invitado Nexus"
  }
]
```

Anexo B: Automatización y Política de Respaldos de Servidor por Replicación Física

Al haber migrado a un almacenamiento basado en ficheros planos del sistema, la estrategia de seguridad perimetral aplica una política de **Copia de Seguridad por Replicación de Archivo Físico**.

```
#!/bin/bash
```

```
DIR_PROYECTO="/var/www/nexus_games_backend"
```

```
DIR_RESPALDO="/var/backups/nexus_db"
```

```
FECHA_ACTUAL=$(date +%F_%H-%M-%S)
```

```
mkdir -p "$DIR_RESPALDO"
```

```
# Ejecución del volcado y compresión tarball de los archivos JSON de datos
```

```
tar -czf "$DIR_RESPALDO/backup_datos_$FECHA_ACTUAL.tar.gz" -C "$DIR_PROYECTO"  
juegos.json comentarios.json
```

```
# Mantenimiento preventivo: eliminar copias con una antigüedad superior a 30 días
```

```
find "$DIR_RESPALDO" -name "backup_datos_*.tar.gz" -mtime +30 -exec rm {} \;
```